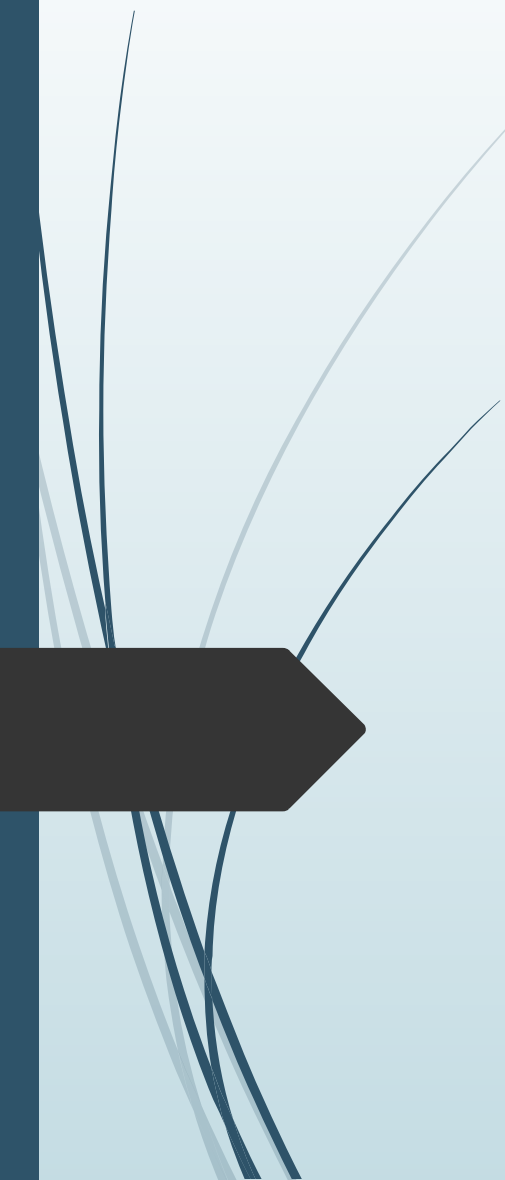


Introduction to Artificial Intelligence

Week 7

Agenda

- MCTS

Monte Carlo Tree Search

Monte Carlo Tree Search

- In computer science, **Monte Carlo tree search** (**MCTS**) is a heuristic search algorithm for some kinds of decision processes, most notably those employed in playing games
- A leading example of Monte Carlo tree search is recent computer Go programs, but it also has been used in other board games, as well as real-time video games and non-deterministic games such as poker





MCTS History

- In 2006, Rémi Coulom described the Monte Carlo method to game-tree search and coined the name Monte Carlo Tree Search
- The Monte Carlo method, which uses random sampling for deterministic problems which are difficult or impossible to solve using other approaches, dates back to the 1940s

Idea

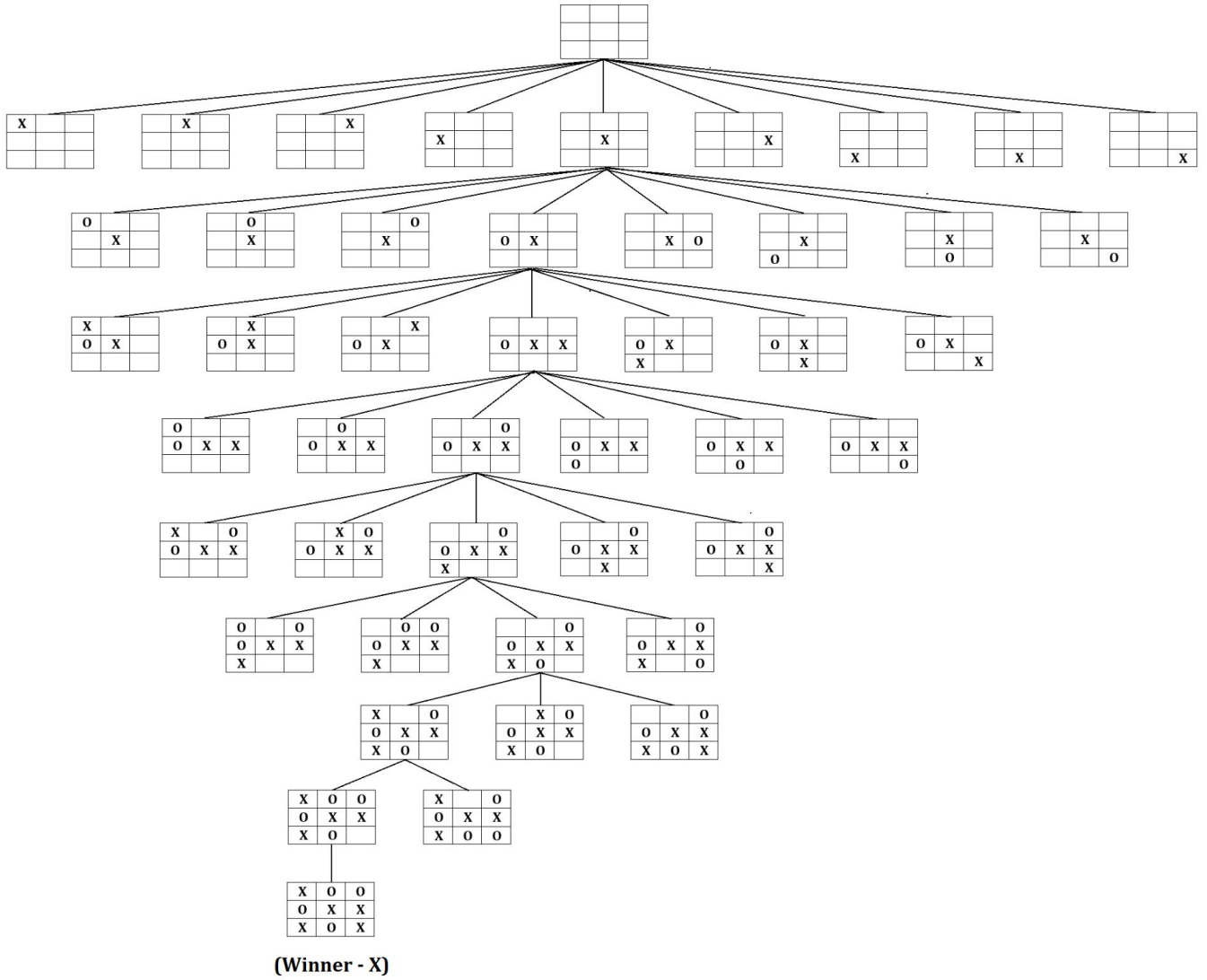
- The focus of MCTS is on the analysis of the most promising moves, expanding the search tree based on random sampling of the search space
- The application of MCTS in games is based on many *playouts* (*rollouts*). In each playout, the game is played out to the very end by selecting moves at random
- The final game result of each playout is then used to weight the nodes in the game tree so that better nodes are more likely to be chosen in future playouts

What makes MCTS different from past decision making algorithms like Minimax?



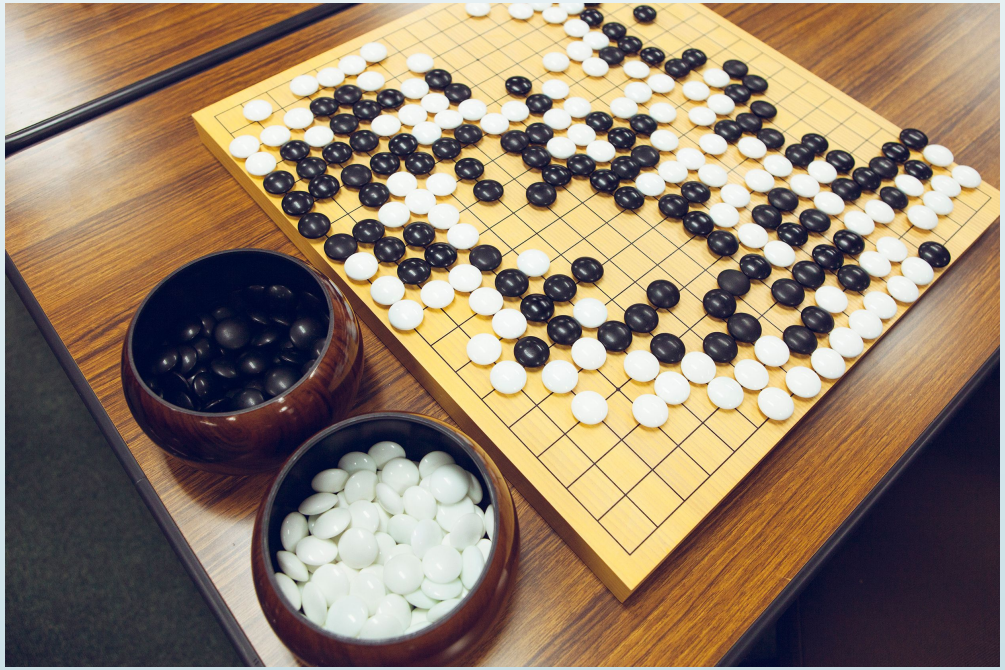
Minimax and Game Trees

- A Game tree is a directed graph whose nodes are positions in a game
- What if game tree is too complex?
  - Higher branching factor

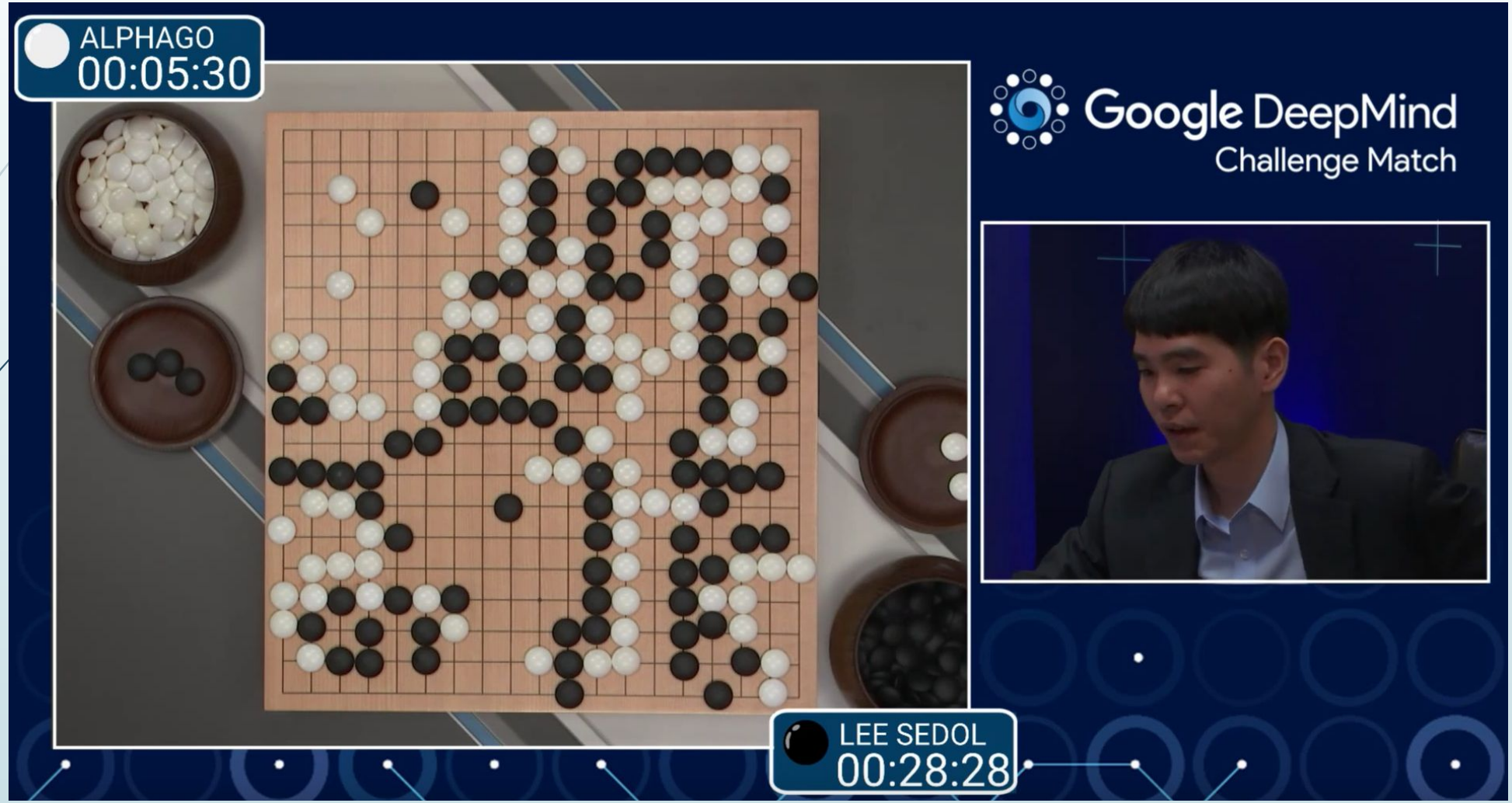


Go

- Go game tree complexity 10^{360} VS chess 10^{123}
- For ~20 years after Deep Blue (chess AI) no AI could master Go



2016 Alpha Go and the MCTS



Monte Carlo Tree Search

- How could it plan ahead when there are so many potential moves in Go?
- BRUTE FORCE minimax method from chess no longer applicable here

Statistics Tree

- MCTS builds a statistics tree (detailing value of nodes) that partially maps onto the entire game tree
- Statistics tree guides the AI to look only/mostly at the most interesting nodes in the game tree
- Value of nodes determined by simulation

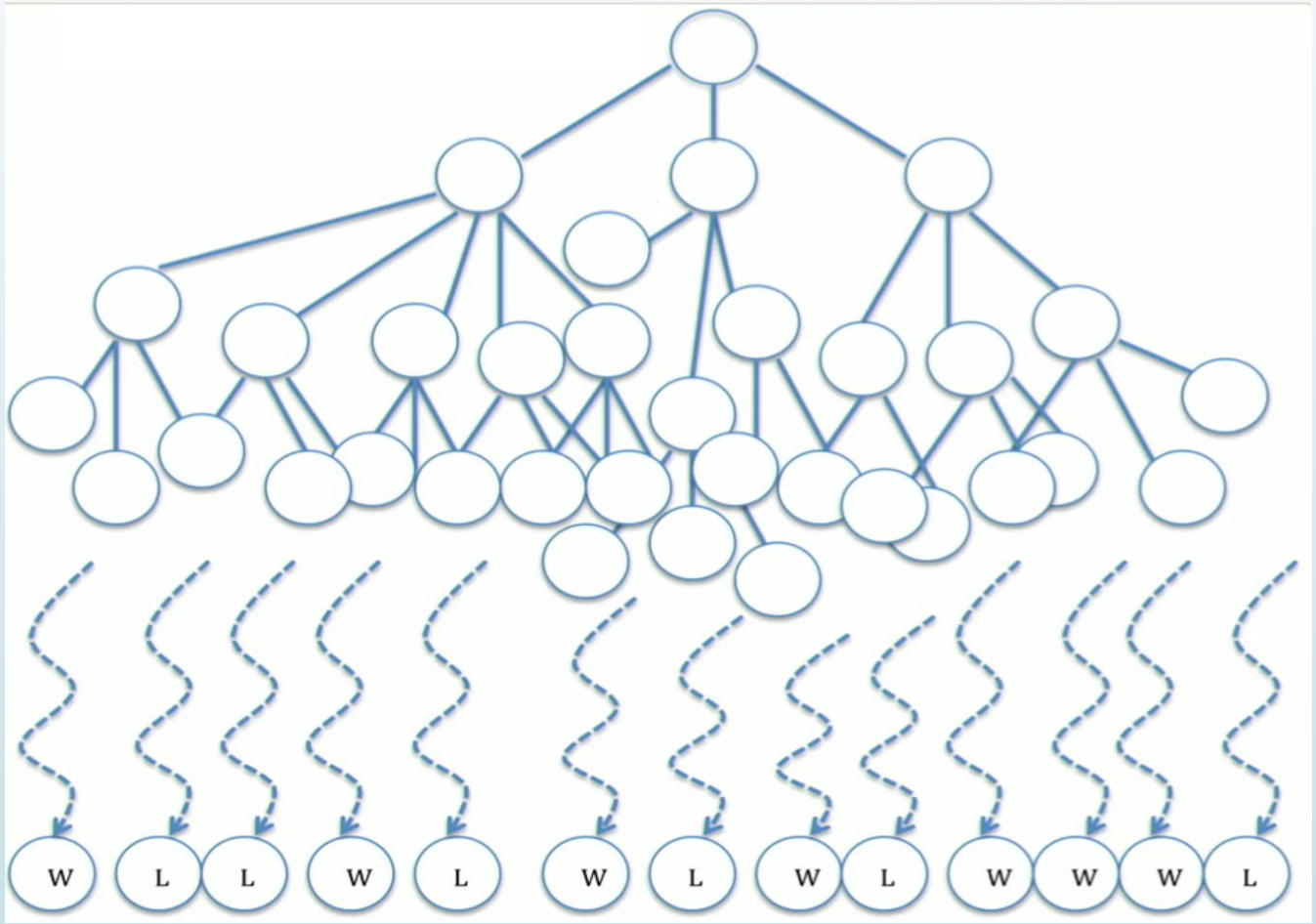
MCTS Steps

- *Pure Monte Carlo Search*
  - The most basic way to use playouts is to apply the same number of playouts after each legal move of the current player, then choosing the move which led to the most victories
  - The efficiency of this method often increases with time as more playouts are assigned to the moves that have frequently resulted in the player's victory (in previous playouts)
  - May work sometimes, but for the most of the games *selection policy* is required
- Full **Monte Carlo Tree Search** employs this principle recursively on many depths of the game tree. Each round of MCTS consists of four steps:
  1. Selection
  2. Expansion
  3. Simulation
  4. Backpropagation

Steps

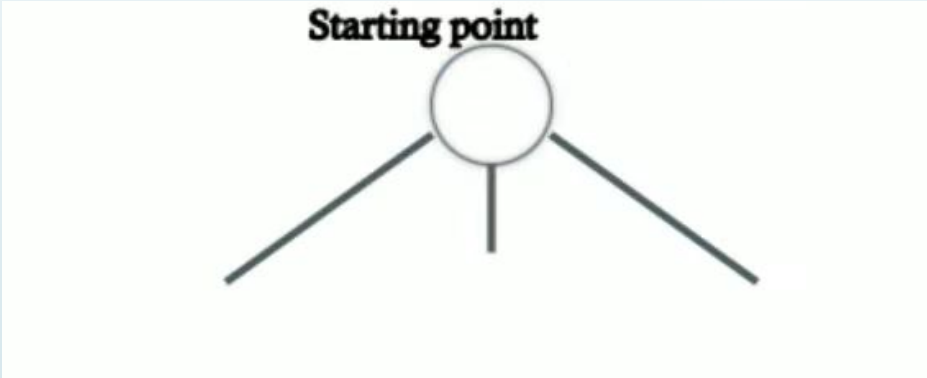
1. Selection
   - Start from root *R* and select successive child nodes down to a leaf node *L*. The section below says more about a way of choosing child nodes that lets the game tree expand towards most promising moves, which is the essence of MCTS
2. Expansion
   - Unless *L* ends the game with a win/loss for either player, either create one or more child nodes or choose from them node *C*
3. Simulation
   - Play a random playout from node *C*.
   - This step is sometimes also called playout or rollout
4. Backpropagation
   - Use the result of the playout to update information in the nodes on the path from *C* to *R*

Hypothetical Game Tree



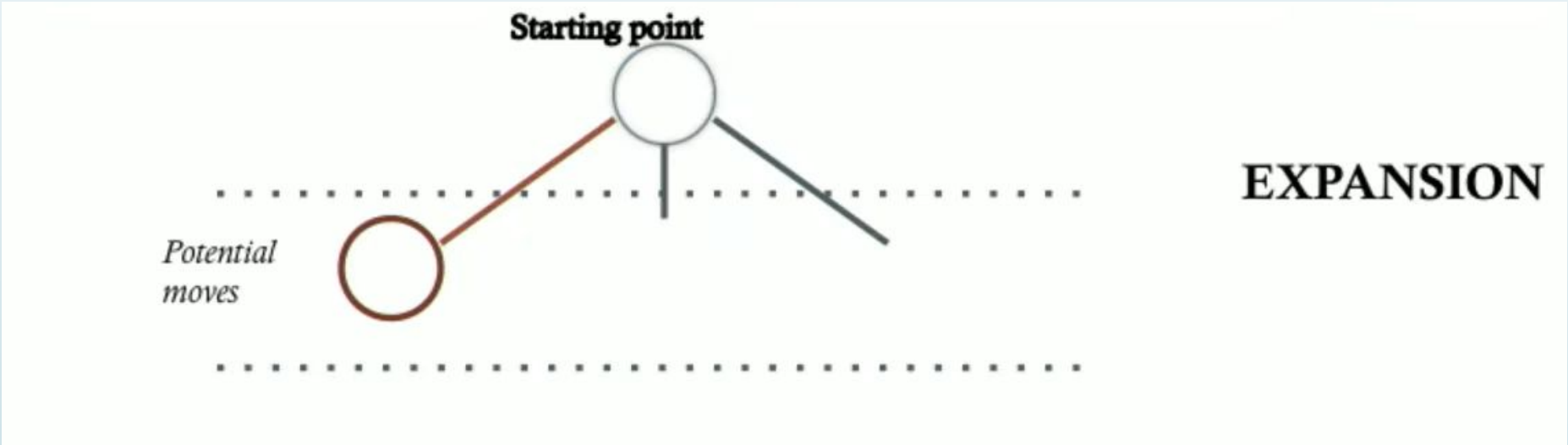
MCTS Example

- MCTS begins its stats tree

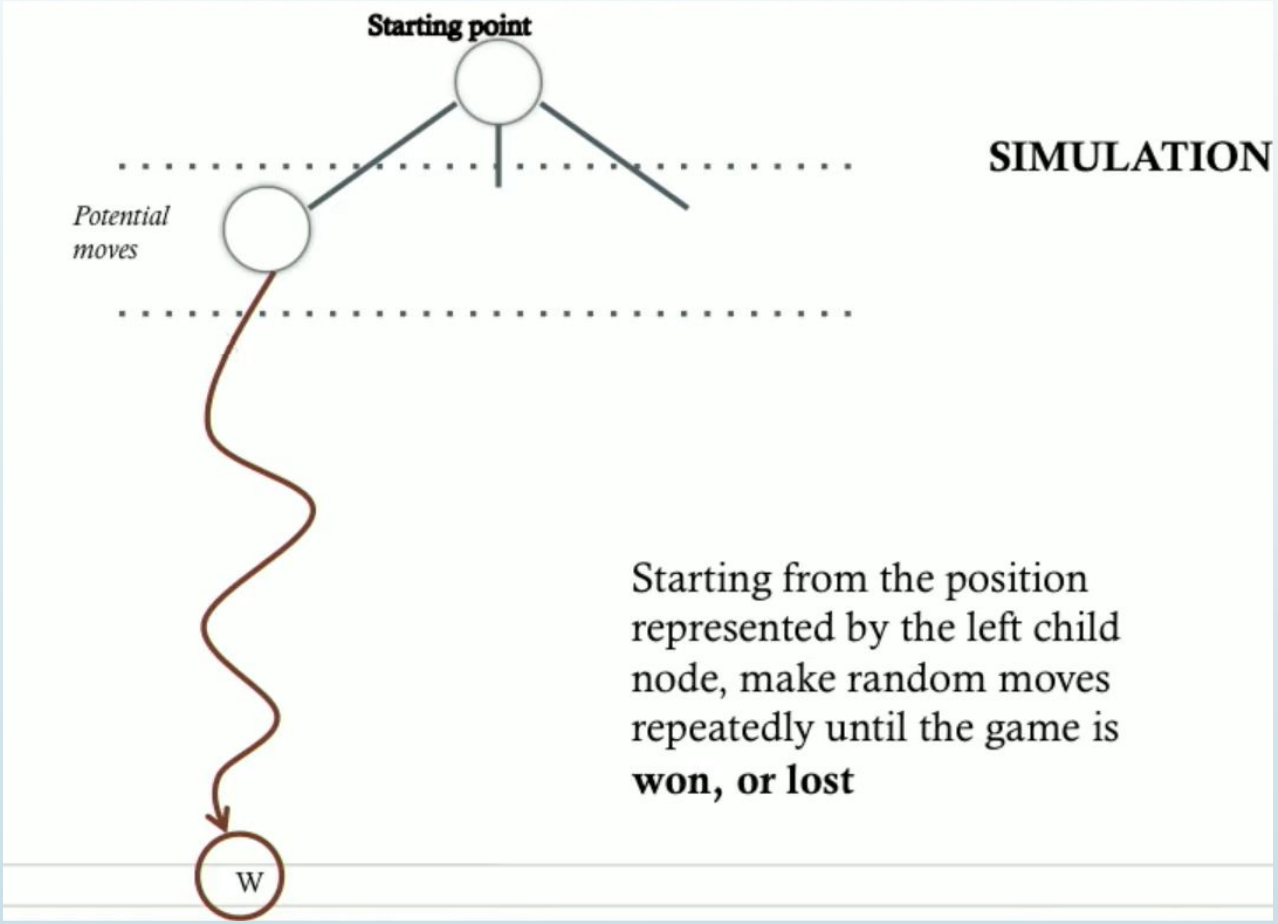


MCTS Example

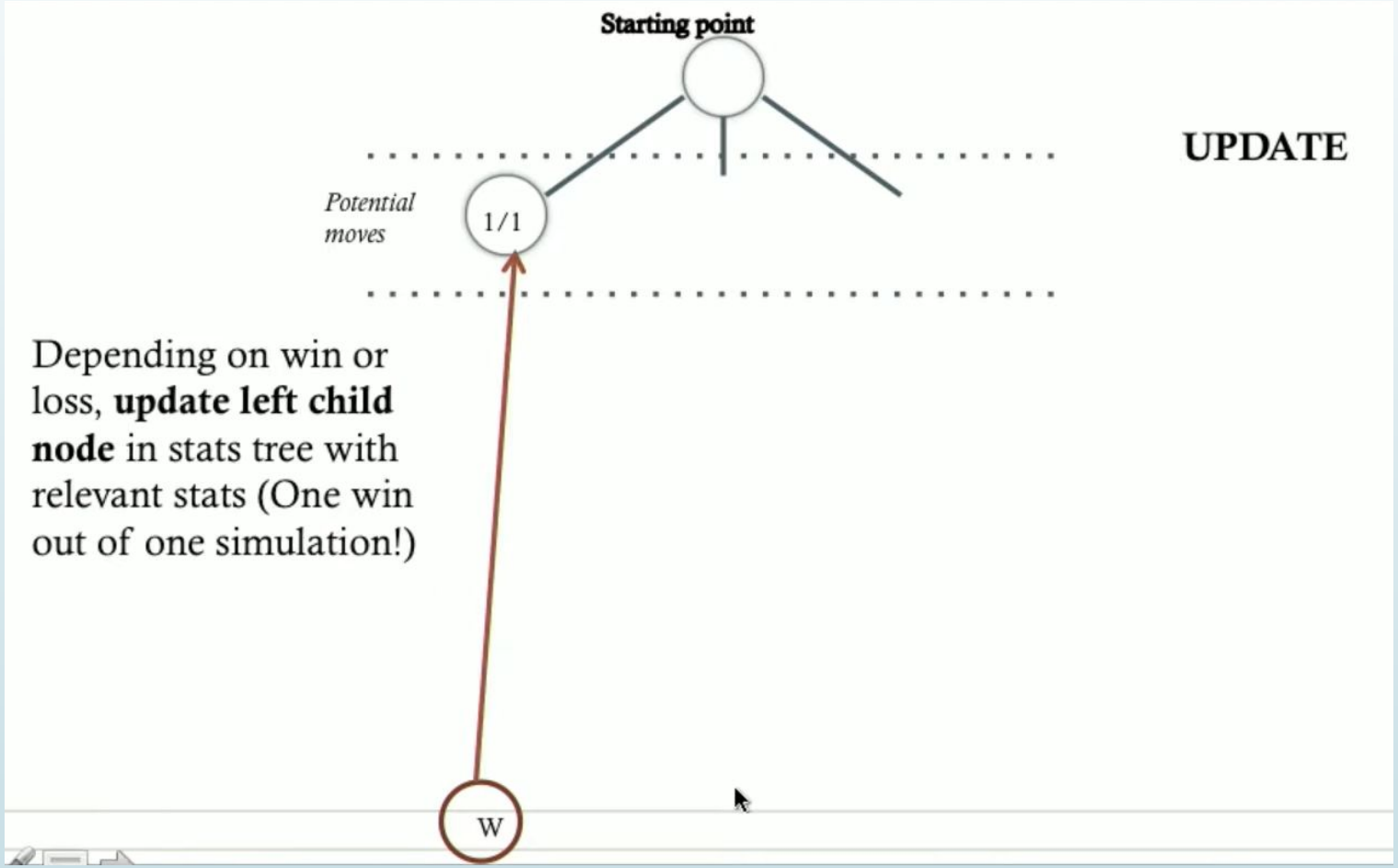
- Add a new node to the stats tree, representing a position in the game tree that the AI will investigate (how good the move is) next



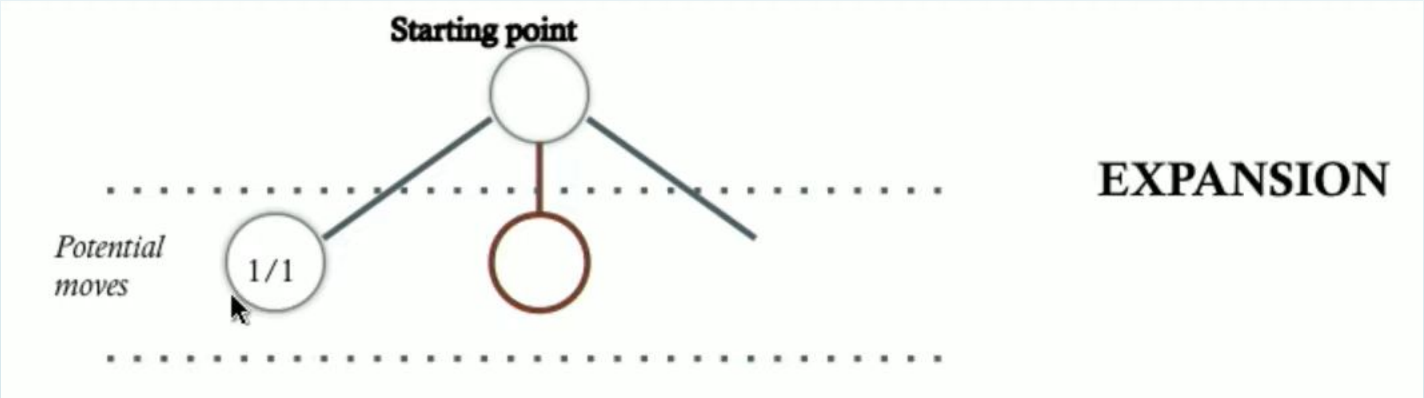
MCTS Example



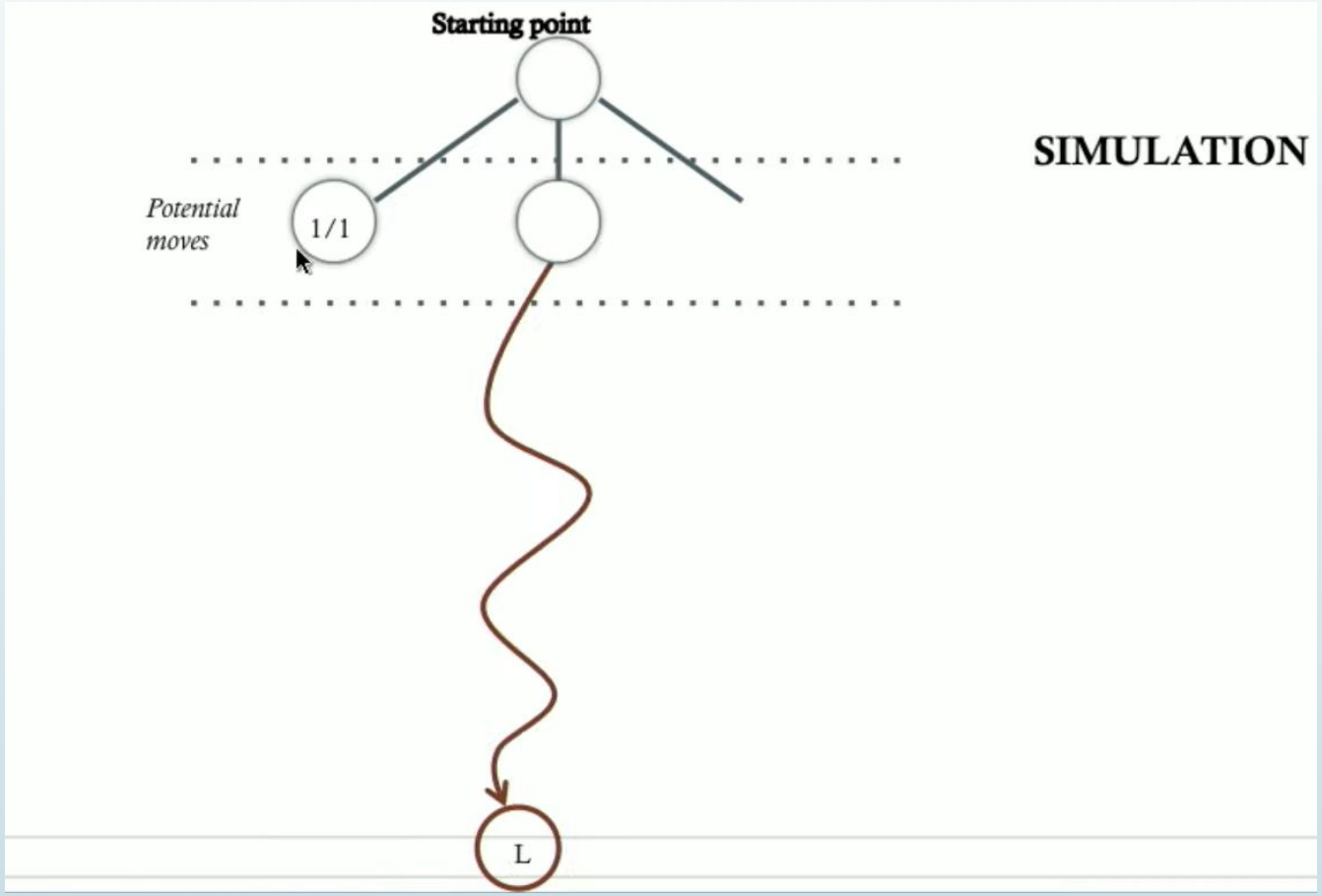
MCTS Example



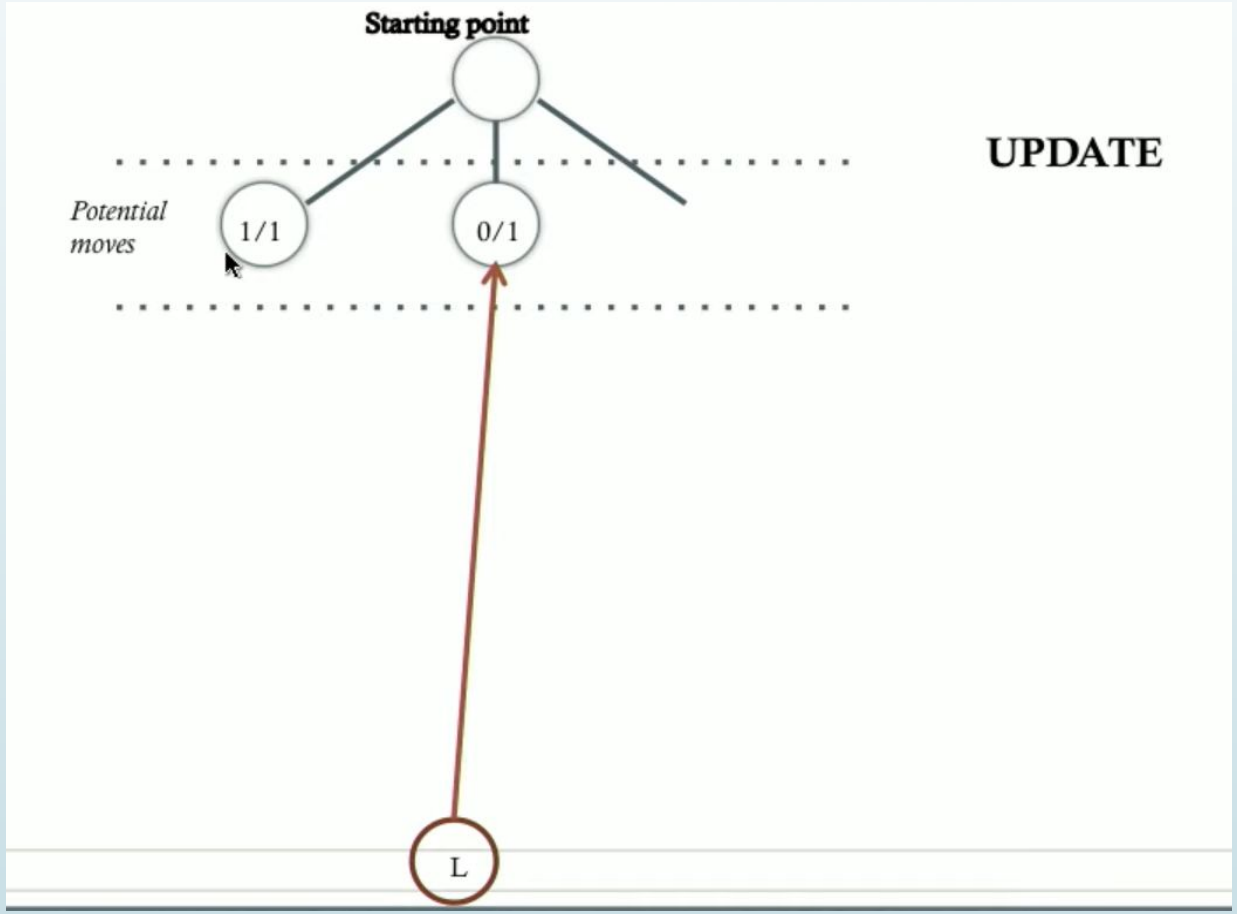
MCTS Example



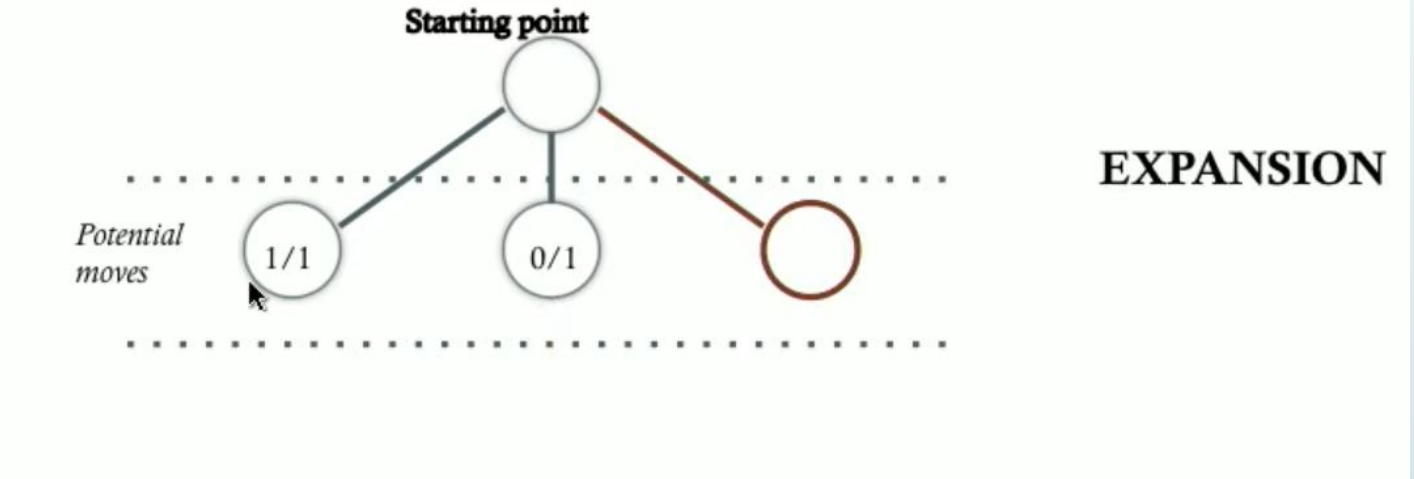
MCTS Example



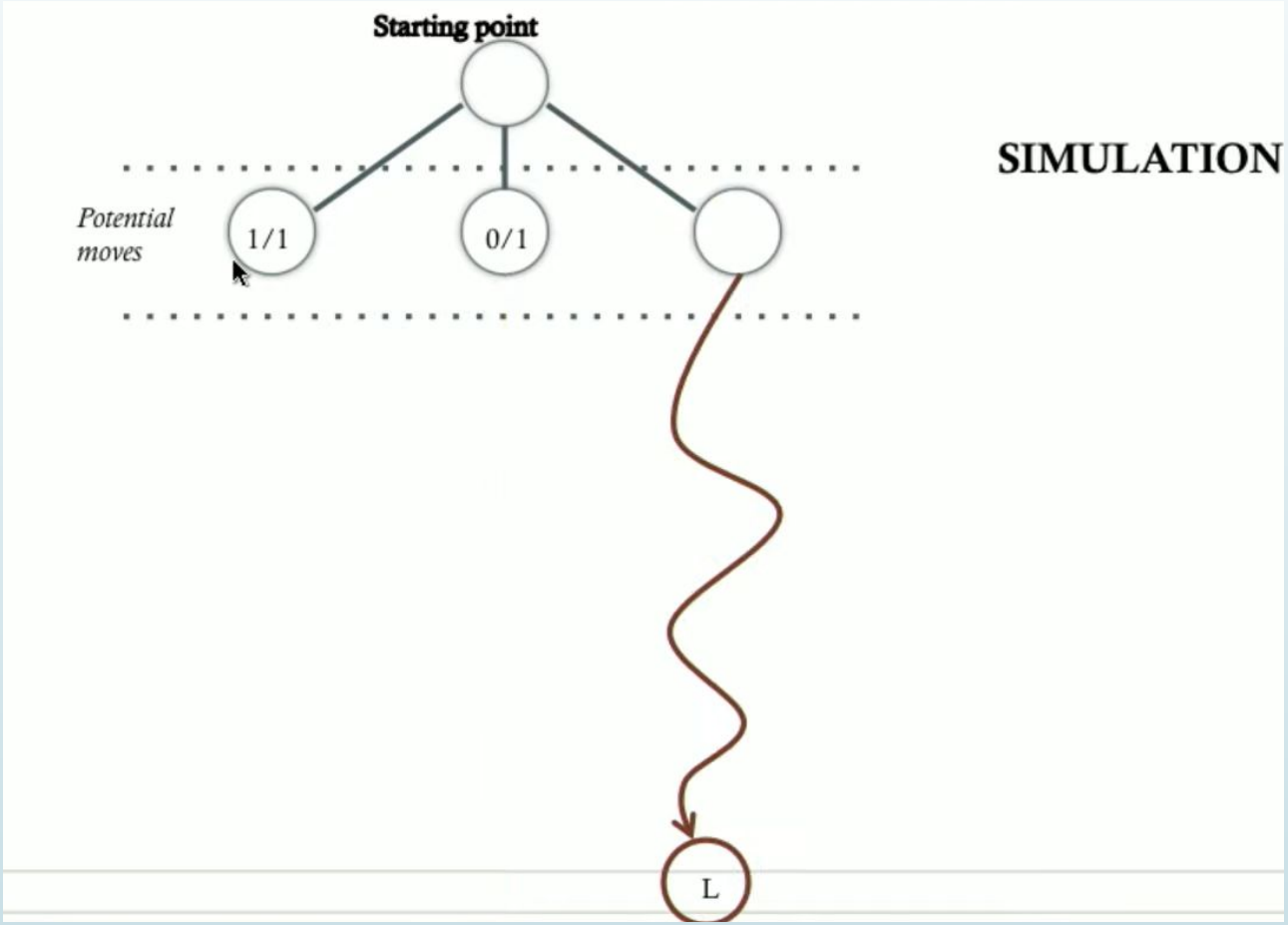
MCTS Example



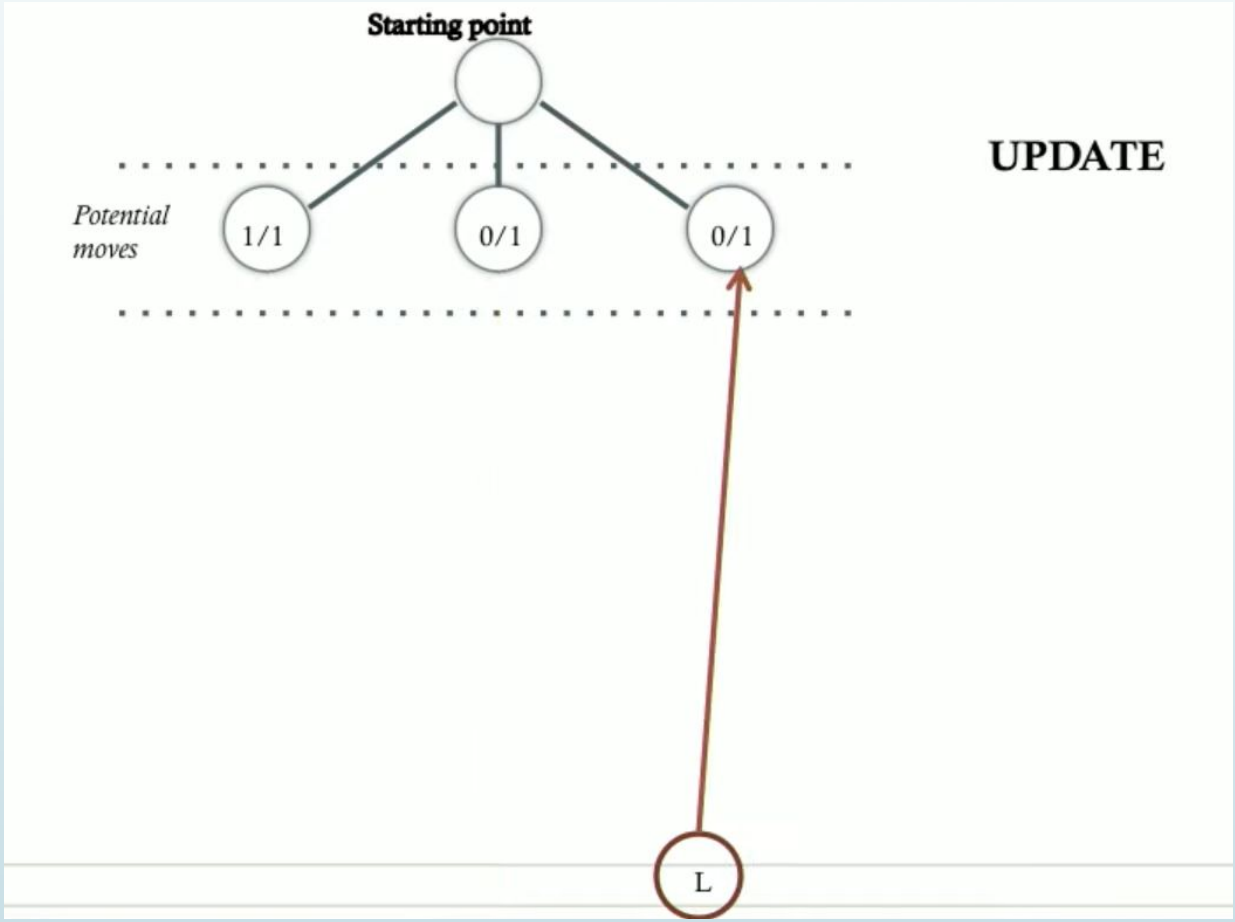
Selection, Expansion, Simulation, Backpropagation



MCTS Example

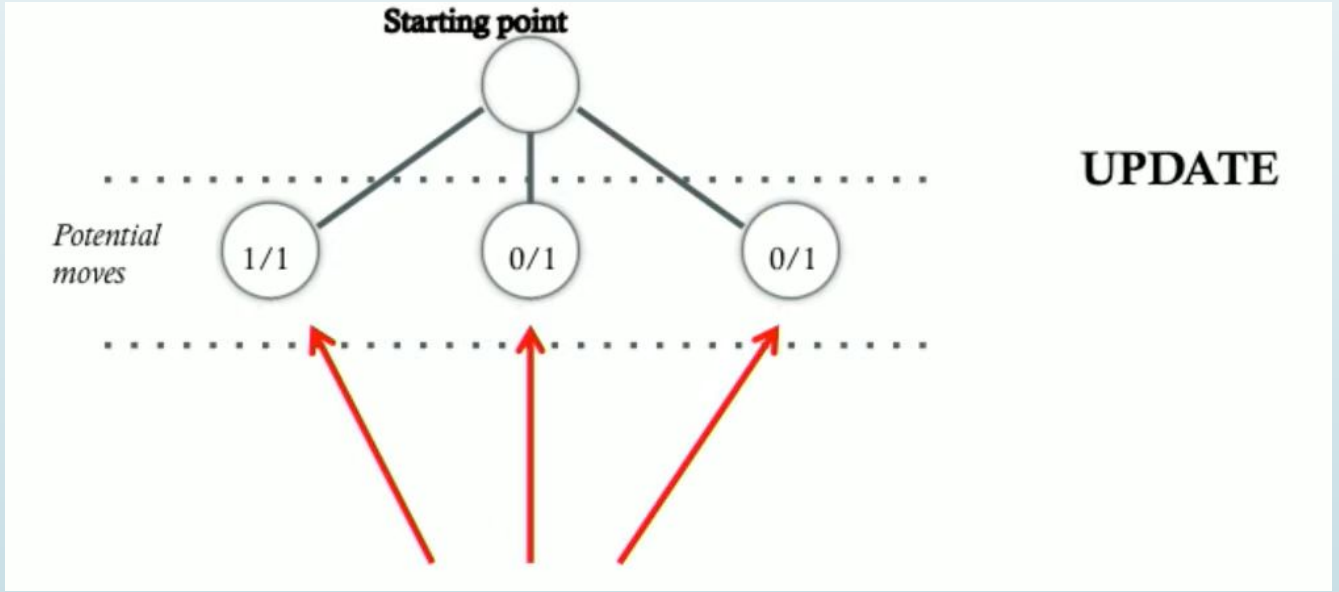


MCTS Example



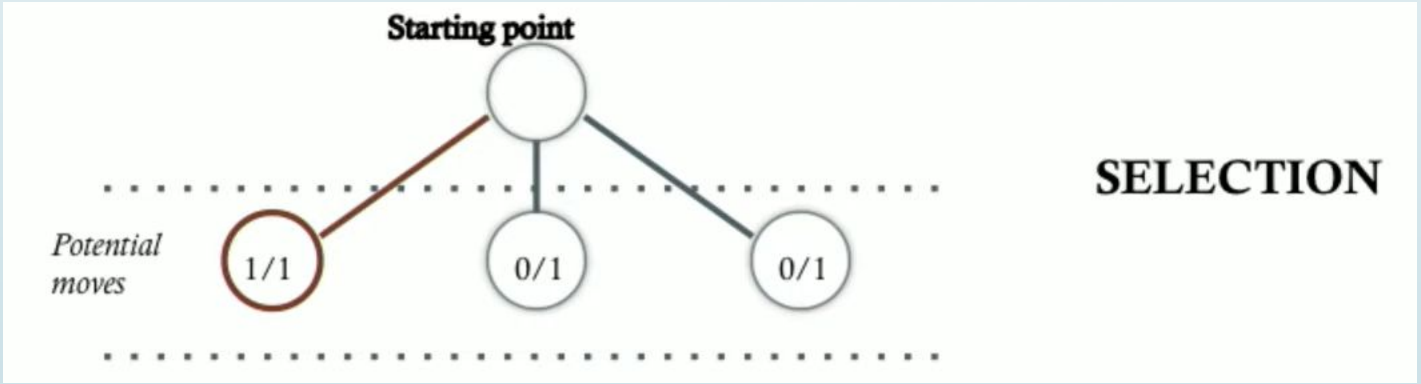
MCTS Example

- Left node seems to be a better move, but chances are, these numbers (produced by a single random simulation) are probably not a good indicator of how good any of the moves are
- More simulations will make them more accurate!



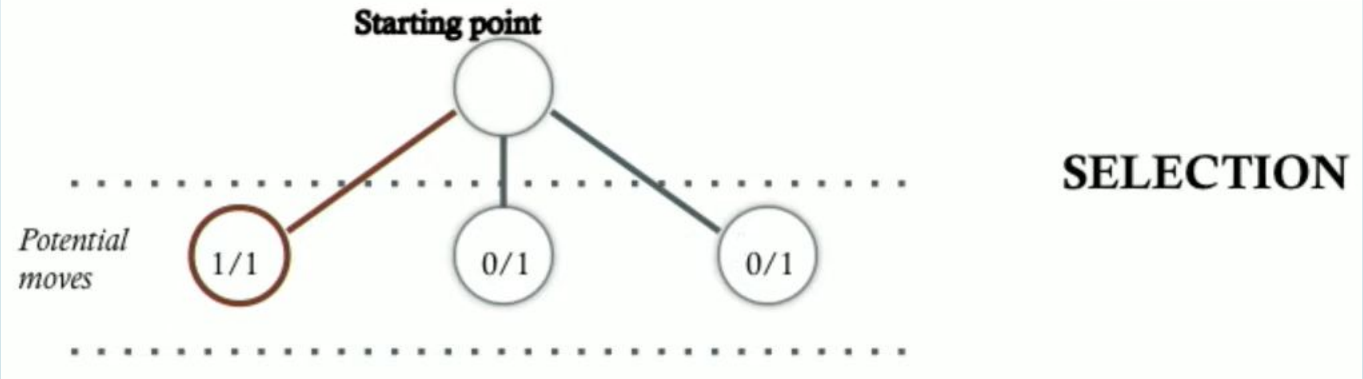
MCTS Example

- All child nodes have now been visited at least once. Now AI can select which child node to be investigated further
- Remember the higher the value, the better the move is (for producing a win)

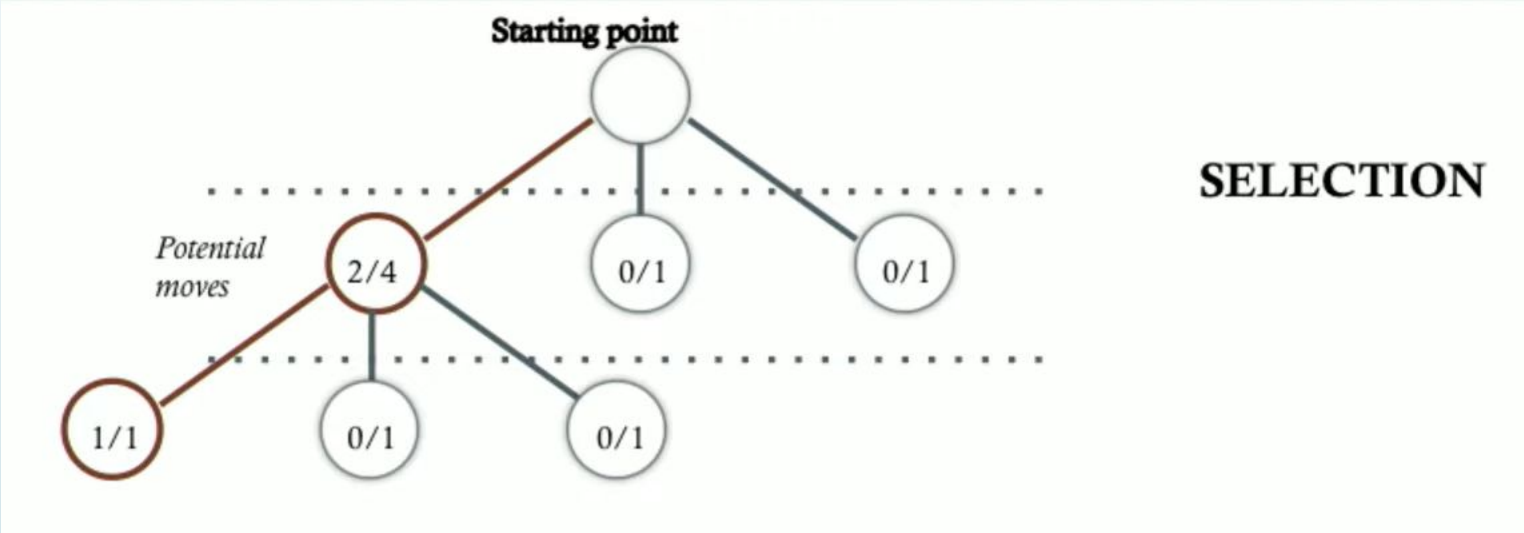


MCTS Example

- Selection is based on 2 things:
  - How good are the statistics?
  - How much has child node been “ignored”

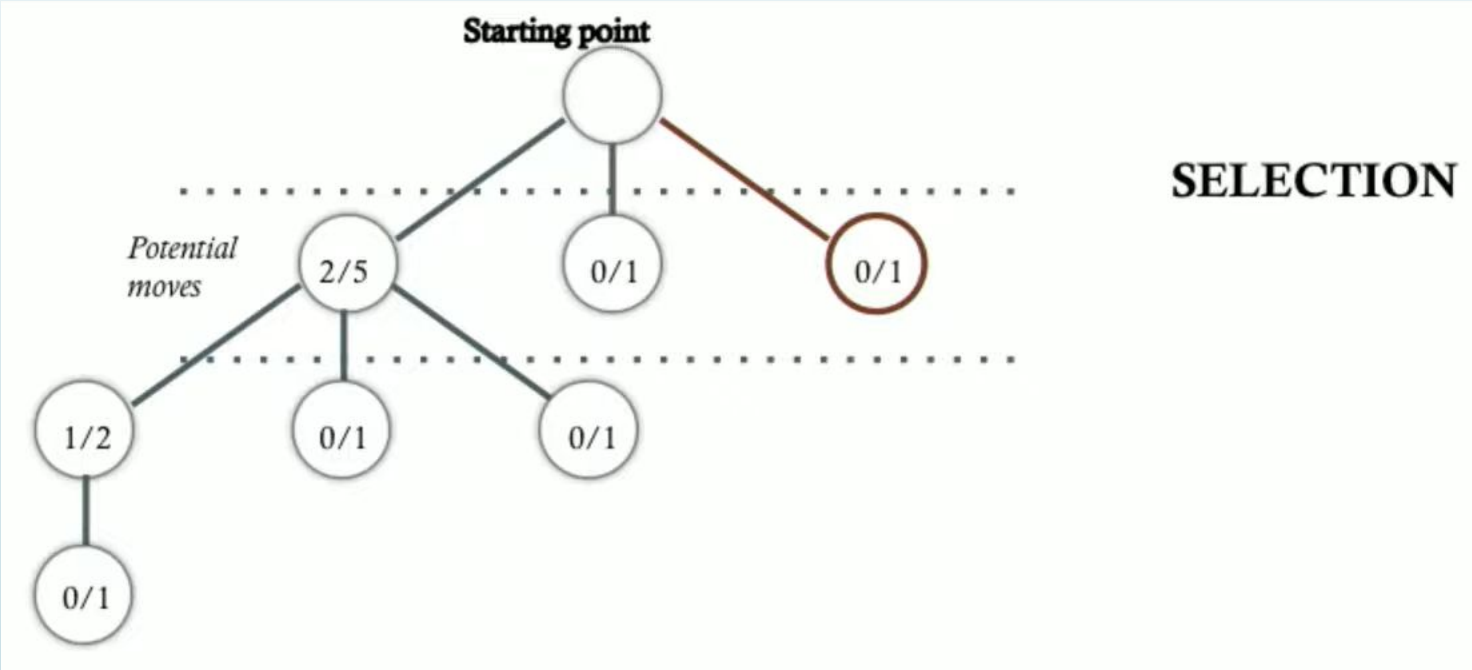


MCTS Example



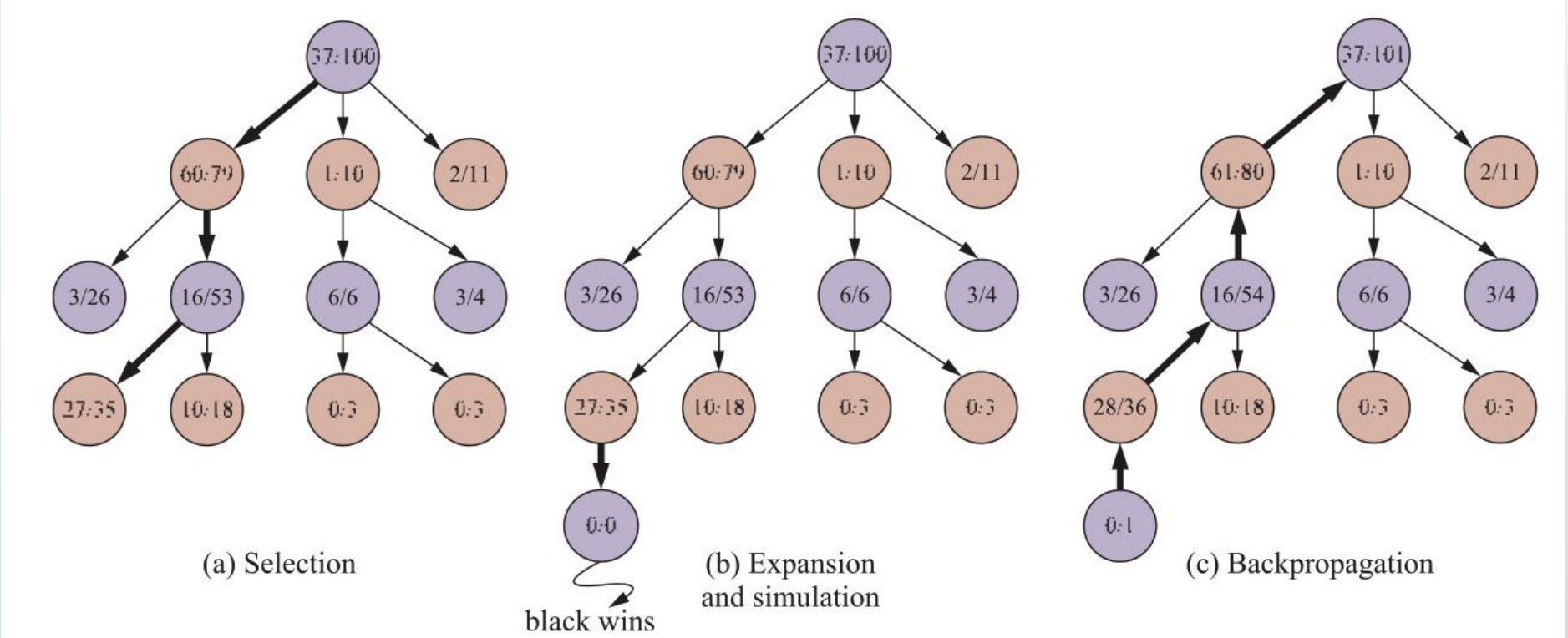
MCTS Example

- These simulations can go on for as long as you want. The more simulation there are, the bigger the stats tree, and the more accurate the node values are



Another MCTS Example

- Two players are involved in this game



What is the Solution?

- Usually, the node with the highest denominator should be the solution
- In other words, the node with the biggest number of simulations
  - You might think that it would be better to return the node with the highest average utility, but the idea is that a node with **65/100 (0.65)** wins is better than one with **2/3 (~0.66)** wins, because the latter has a lot of uncertainty
  - **2/3 (~0.66)** can easily turn into **2/4 (0.5)** with one more rollout
- The number of simulations should be big enough for statistical independence

Exploration vs Exploitation

- The main difficulty in selecting child nodes is maintaining some balance between the *exploitation* of deep variants after moves with high average win rate and the *exploration* of moves with few simulations
- It is recommended to select nodes with the highest value of the next formula called UCT (*Upper Confidence Bound 1 applied to trees*)

$$UCB1 = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}}$$

- Exploitation is the first term: $\frac{w_i}{n_i}$
- Exploration is the second term: $\sqrt{\frac{\ln N_i}{n_i}}$
- c is a constant that balances exploitation and exploration

Exploration vs Exploitation

$$\frac{w_i}{n_i} + c\sqrt{\frac{\ln N_i}{n_i}}$$

- w_i stands for the number of wins for the node considered after the *i*-th move
- n_i stands for the number of simulations for the node considered after the *i*-th move
- N_i stands for the total number of simulations after the *i*-th move run by the parent node of the one considered
- *c* is the exploration parameter — theoretically equal to √2; in practice usually chosen empirically

Advantages and Disadvantages

- Advantages
  - No evaluation function that makes the algorithm ultimate for different tasks
  - Achieves better results than classical algorithms in games with a high branching factor
  - Shortens the search time for large tasks
- Disadvantages
  - In certain positions, there may be moves that look superficially strong, but that actually lead to a loss via a subtle line of play

Parallelization

- Leaf parallelization
  - Parallel execution of many playouts from one leaf of the game tree
- Root parallelization
  - Building independent game trees in parallel and making the move based on the root-level branches of all these trees
- Tree parallelization
  - Parallel building of the same game tree, protecting data from simultaneous writes either with one, global mutex, with more mutexes, or with non-blocking synchronization

Summary

- MCTS

Next week: Midterm Exam

References

- MCTS
- MCTS on Tic-Tac-Toe

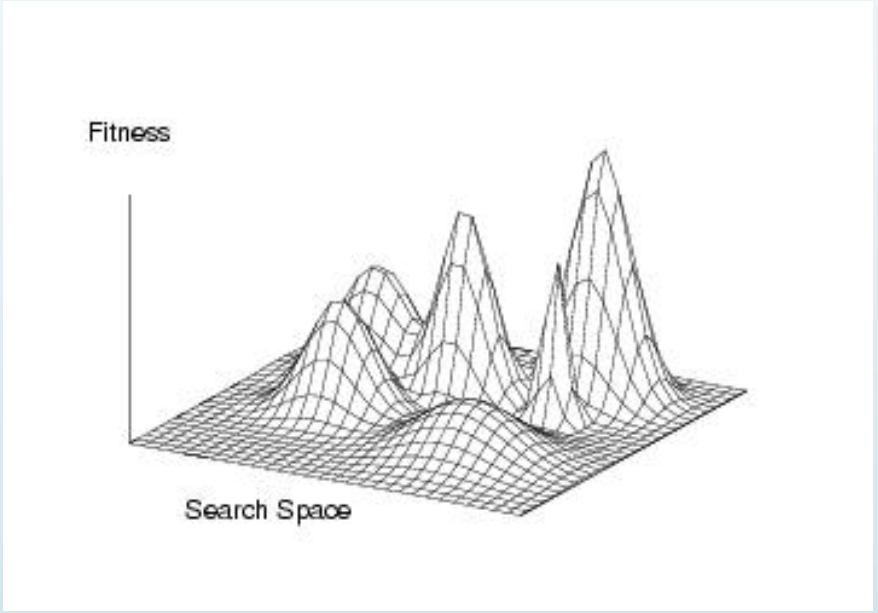
Going up the Mountain… Search as a Landscape

Life is a mountain of solvable problems, and I enjoy that.

James Dyson

Search Space

- We have already heard the term Search Space
- The tuple collection of parameter inputs and outputs for those parameters
- Can we see this as some other shape?



Fitness Landscapes

- Evolutionary biologist Sewall Wright came up with the idea of *fitness landscapes* back in 1932. His maps displayed reproductive success, or fitness of individual organisms as a function of *genotype* (an organism's genetic constitution) or *phenotype* (or organism's observable physical characteristics)

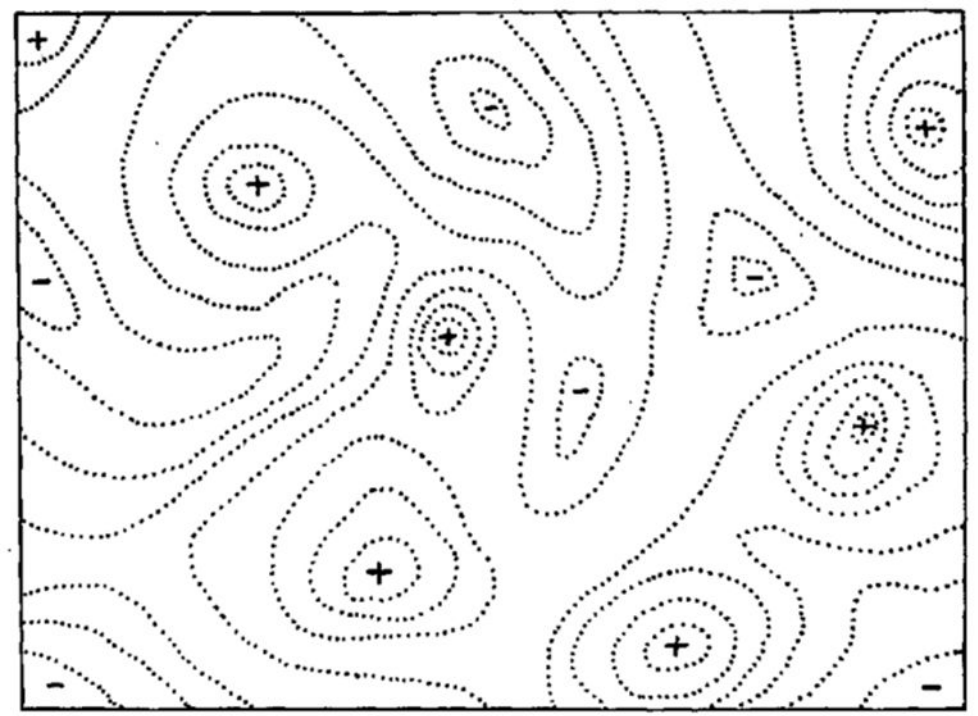


FIGURE 2.—Diagrammatic representation of the field of gene combinations in two dimensions instead of many thousands. Dotted lines represent contours with respect to adaptiveness.

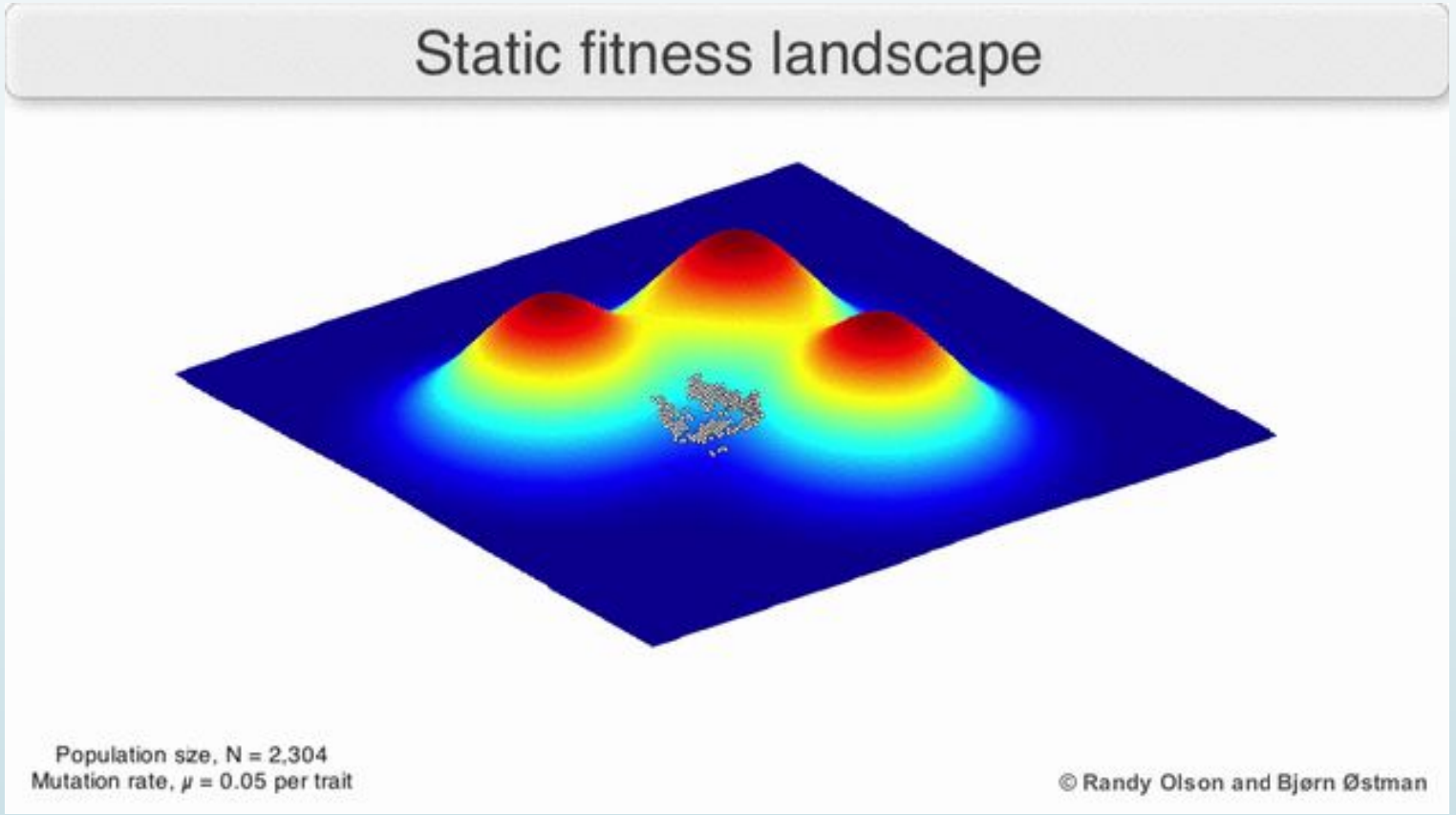
Quantify the Problem

- Is our problem smooth or rough?
  - How smooth or rough?
  - Can we quantify this?
- Do we have gaps?
  - Areas were our parameters
- Can we visualize this?
  - Visuals are helpful to our understanding
  - Demonstrate problems to others

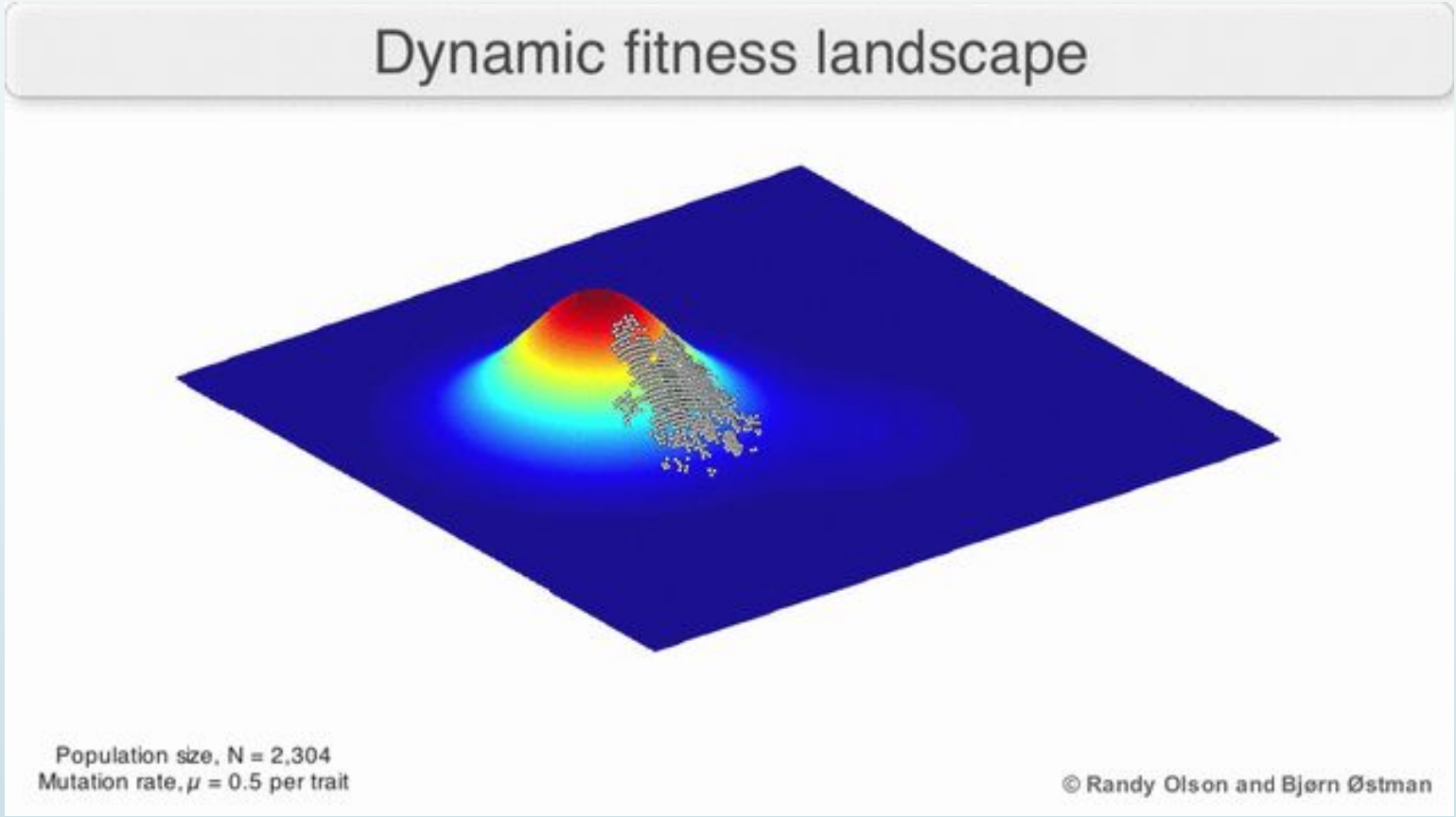
Fitness Landscape

- A fitness landscape is a Tuple $L = (S, f, N)$ where S is the search space of feasible solutions, $f: S \rightarrow R$ is a fitness function, and $N(s)$ is the set of neighbouring points for a member of S such that they can be reached by a single application of the search operation
- This defines a 3-dimensional space which can be projected as a mountain

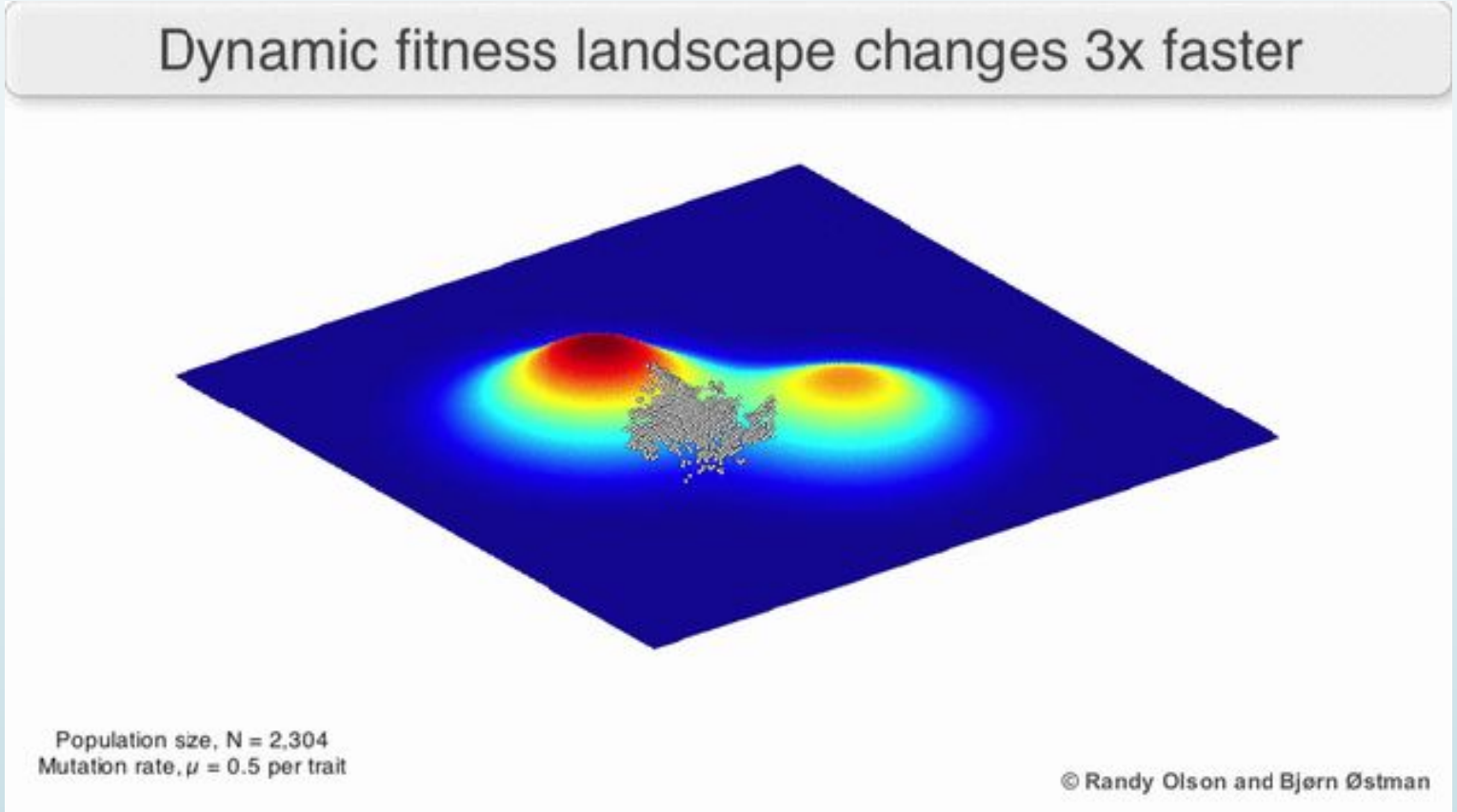
Visualization by R. Olson and Ostman



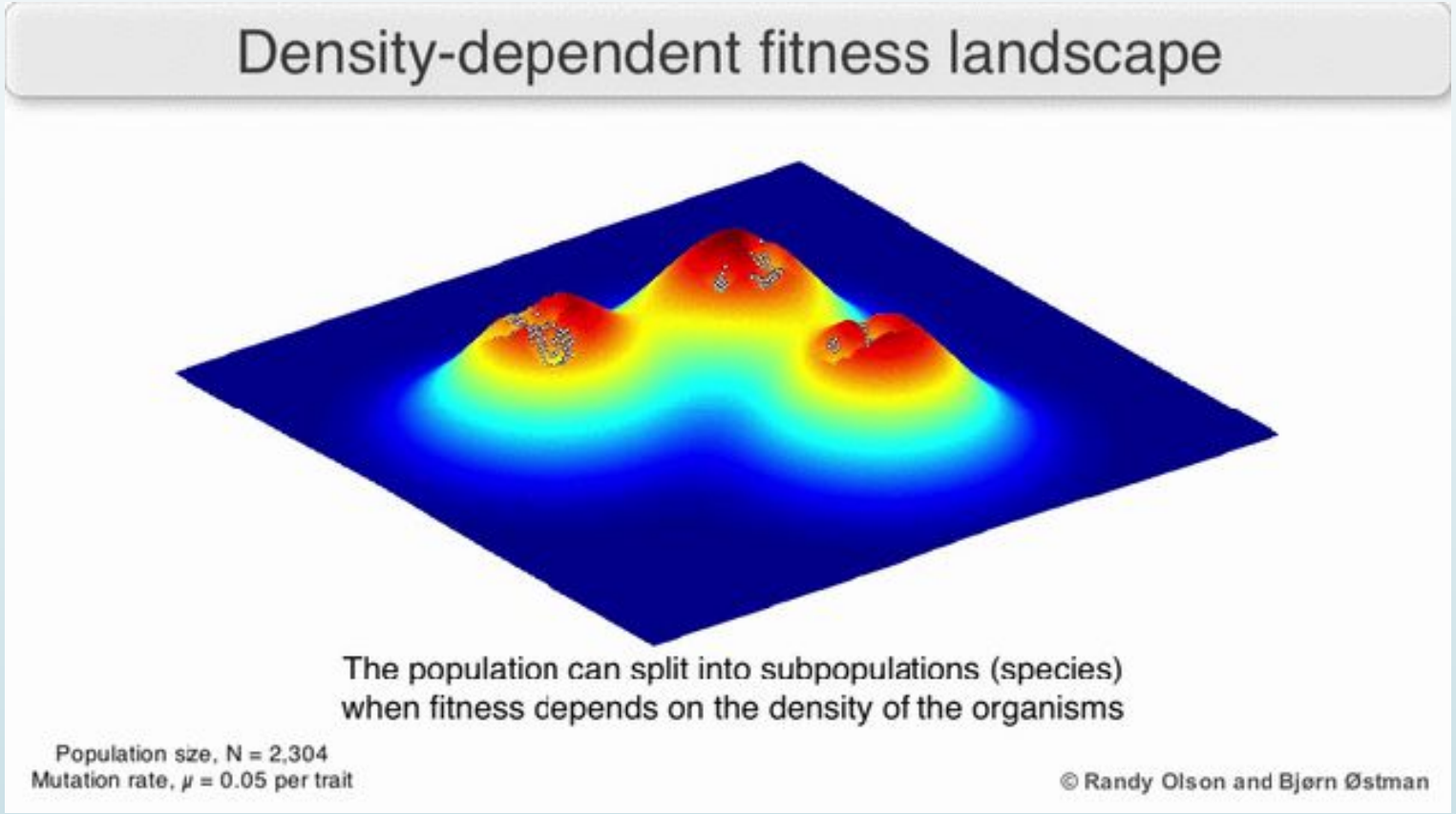
Visualization by R. Olson and Ostman



Visualization by R. Olson and Ostman



Visualization by R. Olson and Ostman



Local Maximums

- A solution *s* in *S* is said to be a local maximum if its fitness is greater than or equal to all of neighbors, i.e. *f(s)>=f(w) V w* in *N(s)*
- If there is a relatively high number of such maximums then the space is called rugged
- If there is a low number of maximums then the space is declared to be smooth
- Note that this depends highly on how the neighbourhood is defined

Basins of Attraction

- A basin of attraction is a set of points which are likely based on their neighbours to move to the same local optimum
- Local Optimums with small basins are considered isolated
- Larger Basins imply a smoother landscape
- If we have few local optimums the space is said to be more neutral

Fitness Neutrality

- If the fitness is relatively neutral then the search will randomly walk about this area
- If this is the case then our movements will for the most part will not progress to a local optimum
- If the space has many local optimum then it may be likely to be stuck, by falling into a pit we cannot move out of based on our neighbor graph

Quantifying the Space

- Take a random walk about the space – we are not interested in the optimization but knowing how the space looks to the operators
- Take the connections mixed with the fitness values, who is a neighbor of who
- Use this measure to get an idea of how the space looks – surveying the space

Information Content

- Uses principles from *Information theory* to give a measure of the ruggedness of the space
- Transform the space or area in a space into a single quantifiable value based on the *Entropy*
- Idea is to make random walks over the space, score them based on the gradient, then check the amount of variation in these changes

Three Modalities

- We move from higher to lower fitness beyond a tolerance
  - Score as a -1
- We keep the same fitness within a tolerance
  - Score as a 0
- We move from a lower to higher fitness beyond a tolerance
  - Score as a 1
- A smooth space will have a very little change in these values (0,0,1,1,1,1,1,1,0)
- A rugged space will have a ‘randomness’ in the values (0,1,-1,-1,0,1,1,0)

Information Content

- Take a number of random walks from p to q, compute the entropy
- For a random walk from p to q
- Pr[pq] – the proportions found of each of the fitness movements on the walks from p to q
- $-\sum_{p \neq q} Pr[pq] \log_6 Pr[pq]$
- Log base 6
  - 0 1
  - 1 0
  - -1 0
  - 0 -1
  - 1 -1
  - -1 1